DiffSTG: Probabilistic Spatio-Temporal Graph Forecasting with Denoising Diffusion Models

Haomin Wen[1], Youfang Lin[1], Yutong Xia[2], Huaiyu Wan[1], Qingsong Wen[3], Roger Zimmermann[2], Yuxuan Liang[4,*]
[1]School of Computer and Information Technology, Beijing Jiaotong University, China
[2]National University of Singapore, [3]DAMO Academy, Alibaba Group
[4]Hong Kong University of Science and Technology (Guangzhou) *corresponding author
{wenhaomin, yflin, hywan}@bjtu.edu.cn; yutong.xia@u.nus.edu;
qingsongedu@gmail.com; rogerz@comp.nus.edu.sg; yuxliang@outlook.com

ABSTRACT

Spatio-temporal graph neural networks (STGNN) have emerged as the dominant model for spatio-temporal graph (STG) forecasting. Despite their success, they fail to model intrinsic *uncertainties* within STG data, which cripples their practicality in downstream tasks for decision-making. To this end, this paper focuses on *probabilistic* STG forecasting, which is challenging due to the difficulty in modeling uncertainties and complex *ST dependencies*. In this study, we present the first attempt to generalize the popular denoising diffusion probabilistic models to STGs, leading to a novel non-autoregressive framework called DiffSTG, along with the first denoising network UGnet for STG in the framework. Our approach combines the spatio-temporal learning capabilities of STGNNs with the uncertainty measurements of diffusion models. Extensive experiments validate that DiffSTG reduces the Continuous Ranked Probability Score (CRPS) by 4%-14%, and Root Mean Squared Error (RMSE) by 2%-7% over existing methods on three real-world datasets. The code is in https://github.com/wenhaomin/DiffSTG.

CCS CONCEPTS

• **Information systems** → **Data mining**; • **Applied computing** → **Forecasting**.

KEYWORDS

Diffusion Model; Probabilistic Forecasting; Spatio-Temporal Graph Forecasting

1 INTRODUCTION

Humans enter a world that is inherently structured, in which a myriad of elements interact with each other both spatially and temporally, resulting in a spatio-temporal composition. Spatio-Temporal Graph (STG) is the de facto most popular tool for injecting such structural information into the formulation of practical problems, especially in smart cities. In this paper, we focus on the problem of *STG forecasting*, i.e., predicting the future signals generated on a graph given its historical observations and the graph structure, such as traffic prediction [17], weather forecasting [31], and taxi demand estimation [45]. To facilitate understanding, a sample illustration is given in Figure 1(a).

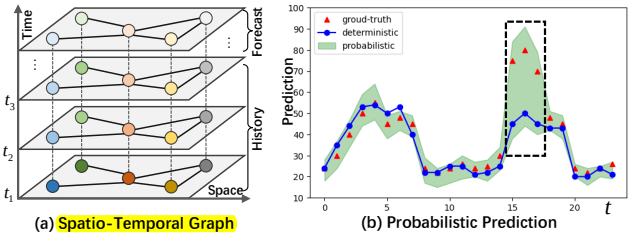


Figure 1: Illustration of probabilistic STG forecasting. (a): Spatio-temporal graph forecasting; (b): Motivation of probabilistic prediction.

Recent techniques for STG forecasting are mostly deterministic, calculating future graph signals exactly without the involvement of randomness. Spatio-Temporal Graph Neural Networks (STGNN) have emerged as the dominant model in this research line. They resort to GNNs for modeling spatial correlations among nodes, and Temporal Convolutional Networks (TCN) or Recurrent Neural Networks (RNN) for capturing temporal dependencies. Though promising, these deterministic approaches still fall short of handling *uncertainties* within STGs, which considerably trims down their practicality in downstream tasks for decision-making. For example, Figure 1(b) depicts the prediction results of passenger flows in a metro station. In the black box, the deterministic method cannot provide the reliability of its predictions. Conversely, the probabilistic method renders higher uncertainties (see the green shadow), which indicates a potential outbreaking of passenger flows in that region. By knowing a range of possible outcomes we may experience and the likelihood of each, the traffic system is able to take operations in advance for public safety management.

While prior endeavors on stochastic STG forecasting were conventionally scarce, we are witnessing a blossom of probabilistic models for time series forecasting [25, 29, 30]. Denoising Diffusion Probabilistic Models (DDPM) [8] are one of the most prevalent

xx, xx, xx

ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

methods in this stream, whose key insight is to produce the future samples by *gradually transforming noise into a plausible prediction through a denoising process*. Unlike vanilla unconditional DDPMs that were originally designed for image generation, such transformation function between consecutive steps is conditioned on the historical time series readings. For example, TimeGrad [25] sets the LSTM-encoded representation of the current time series as the condition, and estimates the future regressively. CSDI [36] directly utilizes observed values as the condition to model the data distribution.

However, the above probabilistic time series models are still insufficient for modeling STGs. Firstly, they only model the temporal dependencies within a single node, without capturing the spatial correlations between different nodes. In reality, objects are correlated with each other spatially, for example, nearby sensors in a road network tend to witness similar traffic trends. Failing to encode such spatial dependencies will drastically deteriorate the predictive accuracy [44, 47]. Secondly, the training and inference of existing probabilistic time series models, e.g., Latent ODE [29] and TimeGrad, suffer notorious inefficiency due to their sequential nature, thereby posing a hurdle to long-term forecasting.

To address these issues, we generalize the popular DDPMs to spatio-temporal graphs for the first time, leading to a novel framework called **DiffSTG**, which couples the spatio-temporal learning capabilities of STGNNs with the uncertainty measurements of DDPMs. Targeting the first challenge, we devise an effective module (UGnet) as the denoising network of DiffSTG. As its name suggests, UGnet leverages a Unet-based architecture [28] to capture multi-scale temporal dependencies and GNN to model spatial correlations. Compared to existing denoising networks in standard DDPMs, our UGnet performs more accurate denoising in the reverse process by virtue of capturing ST dependencies. To overcome the second issue, our DiffSTG produces future samples in a non-autoregressive fashion. In other words, our framework efficiently generates multi-horizon predictions all at once, rather than producing them step by step as what TimeGrad did. In summary, our contributions lie in three aspects:

- We hit the problem of probabilistic STG forecasting from a score-based diffusion perspective with the first shot. Our DiffSTG can effectively model the complex ST dependencies and intrinsic uncertainties within STG data.
- We develop a novel denoising network called UGNet dedicated to STGs for the first time. It contributes as a new and powerful member of DDPMs' denoising network family for modeling ST-dependencies in STG data.
- We empirically show that DiffSTG reduces the Continuous Ranked Probability Score (CRPS) by 4%-14%, and Root Mean Squared Error (RMSE) by 2%-7% over existing probabilistic methods on three real-world datasets.

The rest of this paper is organized as follows. We delineate the concepts of DDPM in Section 2. The formulation and implementation of the proposed DiffSTG are detailed in Section 3 and 4, respectively. We then examine our framework and present the empirical findings in Section 5. Lastly, we introduce related arts in Section 6 and conclude in Section 7.

2 DENOISING DIFFUSION PROBABILISTIC MODELS

Given samples from a data distribution $q(\boldsymbol{x}_0)$, Denoising Diffusion Probabilistic Models (DDPM) [8] are unconditional generative models aiming to learn a model distribution $p_\theta(\boldsymbol{x}_0)$ that approximates $q(\boldsymbol{x}_0)$ and is easy to sample from. Let $\boldsymbol{x}_n$ for $n = 1, \cdots, N$ be a sequence of latent variables from the same sample space of $\boldsymbol{x}_0$ (denoted as $\mathcal{X}$). DDPM are latent variable models of the form $p_\theta(\boldsymbol{x}_0) = \int p_\theta(\boldsymbol{x}_{0:N}) d\boldsymbol{x}_{1:N}$. It contains two processes, namely the forward process and the reverse process.

Forward Process. The forward process is defined by a Markov chain which progressively adds Gaussian noise to the observation $\boldsymbol{x}_0$:

$$q(\boldsymbol{x}_{1:N}|\boldsymbol{x}_0) = \prod_{n=1}^{N} q(\boldsymbol{x}_n|\boldsymbol{x}_{n-1}), \tag{1}$$

where $q(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})$ is a Gaussian distribution as

$$q(\boldsymbol{x}_n|\boldsymbol{x}_{n-1}) = \mathcal{N}(\boldsymbol{x}_n; \sqrt{1-\beta_n}\boldsymbol{x}_{n-1}, \beta_n \mathbf{I}), \tag{2}$$

and $\{\beta_1, \cdots, \beta_N\}$ is an increasing variance schedule with $\beta_n \in (0, 1)$ that represents the noise level at forward step n. Unlike typical latent variable models such as the variational autoencoder [26], the approximate posterior $q(\boldsymbol{x}_{1:N}|\boldsymbol{x}_0)$ in diffusion probabilistic models is not trainable but fixed to a Markow chain depicted by the above Gaussian transition process.

Let $\hat{\alpha}_n = 1 - \beta_n$ and $\alpha_n = \prod_{i=1}^{n} \hat{\alpha}_i$ be the cumulative product of $\hat{\alpha}_n$, a special property of the forward process is that the distribution of $\boldsymbol{x}_n$ given $\boldsymbol{x}_0$ has a close form:

$$q(\boldsymbol{x}_n|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_n; \sqrt{\alpha_n}\boldsymbol{x}_0, (1-\alpha_n)\mathbf{I}), \tag{3}$$

which can also be expressed as $\boldsymbol{x}_n = \sqrt{\alpha_n}\boldsymbol{x}_0 + \sqrt{1-\alpha_n}\boldsymbol{\epsilon}$ by the reparameteriztioin trick [12], with $\boldsymbol{\epsilon} \in \mathcal{N}(\mathbf{0}; \mathbf{I})$ as a sampled noise. The above property allows us to directly sample $\boldsymbol{x}_n$ at any arbitrary noise level n, instead of computing the forward process step by step.

Reverse Process. The reverse process denoises $\boldsymbol{x}_N$ to recover $\boldsymbol{x}_0$ recurrently. It also follows a Markov chain but with learnable Gaussian transitions starting with $p(\boldsymbol{x}_N) = \mathcal{N}(\boldsymbol{x}_N; \mathbf{0}, \mathbf{I})$, which is defined as

$$p_\theta(\boldsymbol{x}_{0:N}) = p(\boldsymbol{x}_N) \prod_{n=N}^{1} p_\theta(\boldsymbol{x}_{n-1}|\boldsymbol{x}_n). \tag{4}$$

Then, the transition between two nearby latent variables is denoted by

$$p_\theta(\boldsymbol{x}_{n-1}|\boldsymbol{x}_n) = \mathcal{N}(\boldsymbol{x}_{n-1}; \mu_\theta(\boldsymbol{x}_n, n), \sigma_\theta(\boldsymbol{x}_n, n)), \tag{5}$$

with shared parameters θ. Here we choose the same parameterization of $p_\theta(\boldsymbol{x}_{n-1}|\boldsymbol{x}_n)$ as in [8] in light of its promising performance on image generation:

$$\mu_\theta(\boldsymbol{x}_n, n) = \frac{1}{\alpha_n}\left(\boldsymbol{x}_n - \frac{\beta_n}{\sqrt{1-\alpha_n}}\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_n, n)\right), \tag{6}$$

$$\sigma_\theta(\boldsymbol{x}_n, n) = \frac{1-\alpha_{n-1}}{1-\alpha_n}\beta_n, \tag{7}$$

where $\boldsymbol{\epsilon}_\theta(\mathcal{X} \times \mathbb{R}) \to \mathcal{X}$ is a trainable denoising function that decides how much noise should be removed at the current denoising step.

The parameters θ are learned by solving the following optimization problem:

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \mathbb{E}_{\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0), \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), n} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\boldsymbol{x}_n, n \right) \right\|_2^2 .$$

Since we already know $\boldsymbol{x}_0$ in the training stage, and recall that $\boldsymbol{x}_n = \sqrt{\alpha_n}\boldsymbol{x}_0 + \sqrt{1-\alpha_n}\boldsymbol{\epsilon}$ by the property as mentioned in the forward process, the above training objective of unconditional generation can be specified as

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \mathbb{E} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\sqrt{\alpha_n}\boldsymbol{x}_0 + \sqrt{1-\alpha_n}\boldsymbol{\epsilon}, n \right) \right\|_2^2 . \tag{8}$$

This training objective can be viewed as a simplified version of loss similar to the one in Noise Conditional Score Networks [34, 35]. Once trained, we can sample $\boldsymbol{x}_0$ from Eq. (4) and Eq. (5) starting from the Guassian noise $\boldsymbol{x}_N$. This reverse process resembles Langevin dynamics, where we first sample from the most noise-perturbed distribution and then reduce the noise scale step by step until we reach the smallest one. We provide details of DDPM in Appendix A.1.

3 DIFFSTG FORMULATION

Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{A}\}$ represent a graph with V nodes, where $\mathcal{V}, \mathcal{E}$ are the node set and edge set, respectively. $\mathbf{A} \in \mathbb{R}^{V \times V}$ is a weighted adjacency matrix to describe the graph topology. For $\mathcal{V} = \{v_1, \ldots, v_V\}$, let $\mathbf{x}_t \in \mathbb{R}^{F \times V}$ denote F-dimentional signals generated by the V nodes at time t. Given historical graph signals $\boldsymbol{x}^{\mathrm{h}} = [\mathbf{x}_1, \cdots, \mathbf{x}_{T_h}]$ of T_h time steps and the graph $\mathcal{G}$ as inputs, STG forcasting aims at learning a function $\mathcal{F}$ to predict future graph signals $\boldsymbol{x}^{\mathrm{p}}$, which is formulated as:

$$\mathcal{F} : (\boldsymbol{x}^{\mathrm{h}}; \mathcal{G}) \rightarrow [\mathbf{x}_{T_h+1}, \cdots, \mathbf{x}_{T_h+T_p}] := \boldsymbol{x}^{\mathrm{p}}, \tag{9}$$

where T_p is the forecasting horizon. In this study, we focus on the task of probabilistic STG forecasting, which aims to estimate the distribution of future graph signals.

As introduced in Section 1, on the one hand, current deterministic STGNNs can capture the spatial-temporal correlation in STG data, while failing to model the uncertainty of the prediction. On the other hand, diffusion-based probabilistic time series forecasting models [25, 36] have powerful abilities in learning high-dimensional sequential data distributions, while incapable of capturing spatial dependencies and facing efficiency problems when applied to STG data.

To this end, we generalize the popular DDPM to spatio-temporal graphs and present a novel framework called DiffSTG for probabilistic STG forecasting in this section. DiffSTG couples the spatio-temporal learning capabilities of STGNNs with the uncertainty measurements of diffusion models.

3.1 Conditional Diffusion Model

The original DDPM is designed to generate an image from a white noise without condition, which is not aligned with our task where the future signals are generated conditioned on their histories. Therefore, for STG forecasting, we first extend the DDPM to a conditional one by making a few modifications to the reverse process. In the unconditional DDPM, the reverse process $p_\theta(\boldsymbol{x}_{0:N})$ in Eq. (4) is used to calculate the final data distribution $q(\boldsymbol{x}_0)$. To get a conditional diffusion model for our task, a natural approach is to add the history $\boldsymbol{x}^{\mathrm{h}}$ and the graph structure $\mathcal{G}$ as the condition in the reverse process in Eq. (4). In this way, the conditioned reverse diffusion process can be expressed as

$$p_\theta(\boldsymbol{x}^{\mathrm{p}}_{0:N} | \boldsymbol{x}^{\mathrm{h}}, \mathcal{G}) = p(\boldsymbol{x}^{\mathrm{p}}_N) \prod_{n=N}^{1} p_\theta(\boldsymbol{x}^{\mathrm{p}}_{n-1} | \boldsymbol{x}^{\mathrm{p}}_n, \boldsymbol{x}^{\mathrm{h}}, \mathcal{G}). \tag{10}$$

The transition probability of two latent variables in Eq. (5) can be extended as

$$\begin{aligned} & p_\theta(\boldsymbol{x}^{\mathrm{p}}_{n-1} | \boldsymbol{x}^{\mathrm{p}}_n, \boldsymbol{x}^{\mathrm{h}}, \mathcal{G}) \\ & = \mathcal{N}(\boldsymbol{x}^{\mathrm{p}}_{n-1}; \mu_\theta(\boldsymbol{x}^{\mathrm{p}}_n, n | \boldsymbol{x}^{\mathrm{h}}, \mathcal{G}), \sigma_\theta(\boldsymbol{x}^{\mathrm{p}}_n, n | \boldsymbol{x}^{\mathrm{h}}, \mathcal{G})). \end{aligned} \tag{11}$$

Furthermore, the training objective in Eq. (8) can be rewritten as a conditional one:

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \mathbb{E}_{\boldsymbol{x}^{\mathrm{p}}_0, \boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\boldsymbol{x}^{\mathrm{p}}_n, n | \boldsymbol{x}^{\mathrm{h}}, \mathcal{G} \right) \right\|_2^2 . \tag{12}$$

3.2 Generalized Conditional Diffusion Model

In Eq. (10)-(12), the condition $\boldsymbol{x}^{\mathrm{h}}$ and denoising target $\boldsymbol{x}^{\mathrm{p}}$ are separated into two sample space $\boldsymbol{x}^{\mathrm{h}} \in \mathcal{X}^{\mathrm{h}}$ and $\boldsymbol{x}^{\mathrm{p}} \in \mathcal{X}^{\mathrm{p}}$. However, they are indeed extracted from two consecutive periods. Here we propose to consider the history $\boldsymbol{x}^{\mathrm{h}}$ and future $\boldsymbol{x}^{\mathrm{p}}$ as a whole, i.e., $\boldsymbol{x}^{\mathrm{all}} = [\boldsymbol{x}^{\mathrm{h}}, \boldsymbol{x}^{\mathrm{p}}] \in \mathbb{R}^{F \times V \times T}$, where $T = T_h + T_p$. The history can be represented by masking all future time steps in $\boldsymbol{x}^{\mathrm{all}}$, denoted by $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}$. So that the condition $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}$ and denoise target $\boldsymbol{x}^{\mathrm{all}}$ share the same sample space $\mathcal{X}^{\mathrm{all}}$. Thus, the masked version of Eq. (10) can be rewritten as

$$p_\theta(\boldsymbol{x}^{\mathrm{all}}_{0:N} | \boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}, \mathcal{G}) = p(\boldsymbol{x}^{\mathrm{all}}_N) \prod_{n=N}^{1} p_\theta(\boldsymbol{x}^{\mathrm{all}}_{n-1} | \boldsymbol{x}^{\mathrm{all}}_n, \boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}, \mathcal{G}). \tag{13}$$

The masked version of Eq. (12) can be rewritten as

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \mathbb{E}_{\boldsymbol{x}^{\mathrm{all}}_0, \boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta \left(\boldsymbol{x}^{\mathrm{all}}_n, n | \boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}, \mathcal{G} \right) \right\|_2^2 . \tag{14}$$

Compared with the formulation in Eq. (10)-(12), this new formulation is a more generalized one which has the following merits. Firstly, the loss in Eq. (14) unifies the reconstruction of the history and estimation of the future, so that the historical data can be fully utilized to model the data distribution. Secondly, the new formulation unifies various STG tasks in the same framework, including STG prediction, generation, and interpolation [15].

Training. In the training process, we first construct the masked signals $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}$ according to observed values. Then, given the conditional masked information $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}$, graph $\mathcal{G}$ and the target $\boldsymbol{x}^{\mathrm{all}}_0$, we sample noise targets $\boldsymbol{x}^{\mathrm{all}}_n = \sqrt{\alpha_n}\boldsymbol{x}^{\mathrm{all}}_0 + \sqrt{1-\alpha_n}\boldsymbol{\epsilon}$, and then train ϵ_θ by the loss function in Eq. (14). The training procedure of DiffSTG is presented in Algorithm 1.

Inference. As outlined in Algorithm 2, the inference process utilizes the trained denoising function $\boldsymbol{\epsilon}_\theta$ to sample $\boldsymbol{x}^{\mathrm{all}}_{n-1}$ step by step according to Eq. (13), under the guidance of $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}$ and $\mathcal{G}$. Note that unlike the previous diffusion-based model, i.e., TimeGrad, which requires executing T_p times of reverse diffusion process for predicting the future T_p steps, DiffSTG only takes one reverse diffusion process to achieve the prediction, thus significantly accelerating the inference speed.

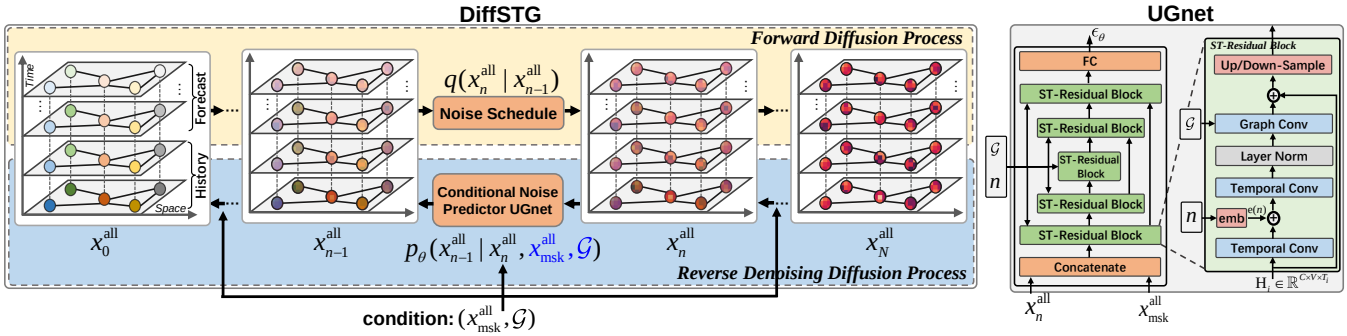


Figure 2: Illustration of proposed DiffSTG and denoising network UGnet. The inference process of DiffSTG utilizes the trained denoising function ϵ_θ (i.e., UGnet) to sample x^{all}_{n-1} step by step, under the guidance of x^{all}_{msk} and $\mathcal{G}$. UGnet leverages an Unet-based architecture to capture multi-scale temporal dependencies and the Graph Neural Network (GNN) to model spatial correlations.

Algorithm 1 Training of DiffSTG

Input: distribution of training data $q(\boldsymbol{x}^{all}_0)$, number of diffusion step N, variance schedule $\{\beta_1, \cdots, \beta_N\}$, graph $\mathcal{G}$.
Output: Trained denoising function $\boldsymbol{\epsilon}_\theta$

1: **repeat**
2: $n \sim \text{Uniform}(\{1, \cdots, N\})$, $\boldsymbol{x}^{all}_0 \sim q(\boldsymbol{x}^{all}_0)$
3: Constructing the masked signals $\boldsymbol{x}^{all}_{msk}$ according to observed values
4: Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ where $\boldsymbol{\epsilon}$'s dimension corresponds to $\boldsymbol{x}^{all}_0$
5: Calculate noisy targets $\boldsymbol{x}^{all}_n = \sqrt{\alpha_n}\boldsymbol{x}^{all}_0 + \sqrt{1-\alpha_n}\boldsymbol{\epsilon}$
6: Take gradient step $\nabla_\theta \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{x}^{all}_n, n | \boldsymbol{x}^{all}_{msk}, \mathcal{G}))\|^2_2$ according to Eq. (14)
7: **until** converged

Algorithm 2 Sampling of DiffSTG

Input: Historical graph signal $\boldsymbol{x}^h$, graph $\mathcal{G}$, trained denoising function $\boldsymbol{\epsilon}_\theta$
Output: Future forecasting $\boldsymbol{x}^p$

1: Construct $\boldsymbol{x}^{all}_{msk}$ according to $\boldsymbol{x}^h$
2: Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ where $\boldsymbol{\epsilon}$'s dimension corresponds to $\boldsymbol{x}^{all}_{msk}$
3: **for** $n = N$ **to** 1 **do**
4: Sample $\boldsymbol{x}^{all}_{n-1}$ using Eq. (13) by taking $\boldsymbol{x}^{all}_{msk}$ and $\mathcal{G}$ as condition
5: **end for**
6: Take out the forecast target in $\boldsymbol{x}^{all}_0$, i.e., $\boldsymbol{x}^p$
7: Return $\boldsymbol{x}^p$

4 DIFFSTG IMPLEMENTATION

After introducing the DiffSTG's formulation, we implement it via elaborately-designed model architecture, which is illustrated in Figure 2. At the heart of the model is the proposed denoising network (UGnet) $\boldsymbol{\epsilon}_\theta$ in the reverse diffusion process, which performs accurate denoising with the ability to effectively model ST dependencies in the data.

4.1 Denoising Network: UGnet

Denoising network $\boldsymbol{\epsilon}_\theta$ in previous works can be mainly classified into two classes, Unet-based architecture [28] for image-related tasks [8, 27, 40], and WaveNet-based architecture [37] for sequence-related tasks [11, 14, 20]. These networks consider the input as either grids or segments, lacking the ability to capture spatio-temporal correlations in STG data. To bridge this gap, we propose a new denoising network $\boldsymbol{\epsilon}_\theta(\mathcal{X}^{all} \times \mathbb{R} | \mathcal{X}^{all}_{msk}, \mathcal{G}) \to \mathcal{X}^{all}$, named UGnet. It adopts an Unet-like architecture in the temporal dimension to capture temporal dependencies at different granularities (e.g., 15 minutes or 30 minutes), and utilizes GNN to model the spatial correlations.

Specifically, as shown in Figure 2, UGnet takes $\boldsymbol{x}^{all}_{msk}$, $\boldsymbol{x}^{all}_n$, n, $\mathcal{G}$ as inputs, and outputs the denoised noise $\boldsymbol{\epsilon}$. It first concatenates $\boldsymbol{x}^{all}_n \in \mathbb{R}^{F \times V \times T}$ and $\boldsymbol{x}^{all}_{msk} \in \mathbb{R}^{F \times V \times T}$ in the temporal dimension to form a new tensor $\widetilde{\boldsymbol{x}}^{all}_n \in \mathbb{R}^{F \times V \times 2T}$, which is projected to a high-dimensional representation $\mathbf{H} \in \mathbb{R}^{C \times V \times 2T}$ by a linear layer, where C is the projected dimension. Then $\mathbf{H}$ is fed into several Spatio-temporal Residual Blocks (ST-Residual Blocks for short), each capturing temporal and spatial dependencies, respectively. Let $\mathbf{H}_i \in \mathbb{R}^{C \times V \times T_i}$ (where $\mathbf{H}_0 = \mathbf{H}$) denote the input of the i-th ST-Residual Block, where T_i is the length of time dimension.

Temporal Dependency Modeling. As shown in Figure 8, at each ST-Residual Block, $\mathbf{H}_i$ is fed into a Temporal Convolution Network (TCN) [1] for modeling temporal dependence, which is a 1-D gated causal convolution of K kernel size with padding to get the same shape with input. The convolution kernel $\Gamma_{\mathcal{T}} \in \mathbb{R}^{K \times C^t_{in} \times C^t_{out}}$ maps the input $\mathbf{H}_i$ to outputs $\mathbf{P}_i, \mathbf{Q}_i \in \mathbb{R}^{C^t_{out} \times V \times T_i}$ with the same shape. Formally, the temporal gated convolution can be defined as

$$\Gamma_{\mathcal{T}}(\mathbf{H}_i) = \mathbf{P}_i \odot \sigma(\mathbf{Q}_i) \in \mathbb{R}^{C^t_{out} \times V \times T_i}, \tag{15}$$

where $\odot$ is the element-wise Hadamard product, and σ is the sigmoid activation function. The item $\sigma(\mathbf{Q}_i)$ can be considered a gate that filters the useful information of $\mathbf{P}_i$ into the next layer. We denote the output of TCN as $\overline{\mathbf{H}}_i$.

Spatial Dependency Modeling. Graph Convolution Networks (GCNs) are generally employed to extract highly meaningful features in the space domain [49]. The graph convolution can be formulated as

$$\Gamma_{\mathcal{G}}(\overline{\mathbf{H}}_i) = \sigma\left(\Phi\left(\mathbf{A}_{gcn}, \overline{\mathbf{H}}_i\right)\mathbf{W}_i\right), \tag{16}$$

where $\mathbf{W}_i \in \mathbb{R}^{C^g_{in} \times C^g_{in}}$ denotes a trainable parameter and σ is an activation function. $\Phi(\cdot)$ is an aggregation function that decides the rule of how neighbors' features are aggregated into the target node. In this work, we do not focus on developing the function $\Phi(\cdot)$. Instead,

we use the form in the most popular vanilla GCN [13] that defines a symmetric normalized summation function as $\Phi_{\text{gcn}}\left(\mathbf{A}_{\text{gcn}}, \overline{\mathbf{H}}_i\right) = \mathbf{A}_{\text{gcn}}\overline{\mathbf{H}}_i$, where $\mathbf{A}_{\text{gcn}} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{A}+\mathbf{I})\mathbf{D}^{-\frac{1}{2}} \in \mathbb{R}^{V\times V}$ is a normalized adjacent matrix of graph $\mathcal{G}$. $\mathbf{I}$ is the identity matrix and $\mathbf{D}$ is the diagonal degree matrix with $\mathbf{D}_{ii} = \sum_j (\mathbf{A}+\mathbf{I})_{ij}$. Note that we reshape the output of the TCN layer to $\overline{\mathbf{H}}_i \in \mathbb{R}^{V\times C^{\text{g}}_{\text{in}}}$, where $C^{\text{g}}_{\text{in}} = T_i \times C^{\text{t}}_{\text{out}}$, and fed this node feature $\overline{\mathbf{H}}_i$ to GCN.

Noise Level Embedding. As shown in the right part of Figure 2, like previous diffusion-based models [25], we use positional encodings of the noise level $n \in [1, N]$ and process it using a transformer positional embedding [38]:

$$\mathbf{e}(n) = \left[\ldots, \cos\left(n/r^{\frac{-2d}{D}}\right), \sin\left(n/r^{\frac{-2d}{D}}\right), \ldots\right]^{\text{T}}, \tag{17}$$

where $d = 1, \cdots, D/2$ is the dimension number of the embedding (set to 32), and r is a large constant 10000. For more details about UGnet, please refer to Appendix A.2.

We highly the unique characteristics and our special design of UGnet by making comparisons with the popular Unet used in diffusion models for the computer version. 1) U-Net is designed for extracting semantic features from Euclidean data, such as images, while UGNet is designed for non-Euclidean STG data to model the spatial-temporal correlations. 2) The U-structure in U-Net serves to aggregate features in the spatial dimension to extract high-level semantic features in the image, while the U-structure in UGNet aggregates signals in the time dimension to capture time dependencies at different granularities. 3) U-Net utilizes 2D-CNNs to capture the spatial correlation, whereas UGNet utilizes graph convolution to model the spatial correlation, which is more suitable for graph-based data. In summary, the proposed UGnet goes beyond a simple replacement of U-Net by changing the input from image to graph $\mathcal{G}$, and involves careful design considerations for STG data.

4.2 Sampling Acceleration

From a variational perspective, a large N (e.g., $N = 1000$ in [8]) allows results of the forward process to be close to a Gaussian distribution so that the reverse denoise process started with Gaussian distribution becomes a good approximation. However, large N makes the sampling low-efficiency since all N iterations have to be performed sequentially. To accelerate the sampling process, we adopt the sampling strategy in [33], which only samples a subset $\{\tau_1, \cdots, \tau_M\}$ of M diffusion steps. Formally, the accelerated sampling process can be denoted as

$$\begin{aligned}\boldsymbol{x}_{\tau_{m-1}} = \sqrt{\alpha_{\tau_{m-1}}}\left(\frac{\boldsymbol{x}_{\tau_m} - \sqrt{1-\alpha_{\tau_m}}\boldsymbol{\epsilon}_\theta^{(\tau_m)}}{\sqrt{\alpha_{\tau_m}}}\right) + \\ \sqrt{1-\alpha_{\tau_{m-1}}-\sigma^2_{\tau_m}}\cdot\boldsymbol{\epsilon}_\theta^{(\tau_m)} + \sigma_{\tau_m}\boldsymbol{\epsilon}_{\tau_m},\end{aligned} \tag{18}$$

where $\boldsymbol{\epsilon}_{\tau_m} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is standard Gaussian noise independent of $\boldsymbol{x}_n$. And σ_{τ_m} controls how stochastic the denoising process is. We set $\sigma_n = \sqrt{(1-\alpha_{n-1})/(1-\alpha_n)}\sqrt{1-\alpha_n/\alpha_{n-1}}$ for all diffusion steps, to make the generative process become a DDPM. When the length of the sampling trajectory is much smaller than N, we can achieve significant increases in computational efficiency. Moreover, note that the data in the last k few reverse steps $\boldsymbol{x}_i^{\text{all}}$ ($i \in \{1, \ldots, k\}$) can be considered a good approximation of the target. Thus we can also add them as samples, reducing the number of the reverse diffusion process from S to S/k, where S is the required sample number to form the data distribution.

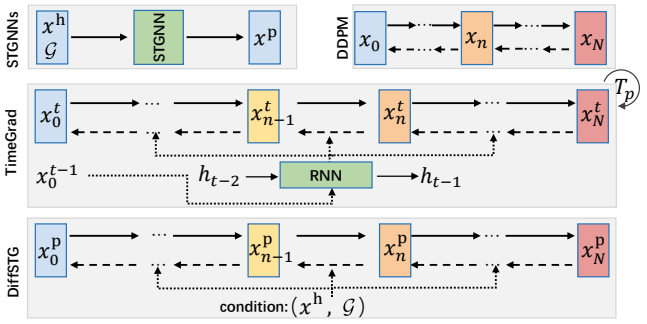


Figure 3: Overview of different models.

4.3 Comparsion among Different Approaches

We give the overview of related models in Figure 3: i) Deterministic STGNNs calculate future graph signals exactly without the involvement of randomness. While the vanilla DDPM is a latent variable generative model without condition; ii) To estimate the data distribution from a trained model, i.e., getting S samples, TimeGrad runs $S \times T_p \times N$ diffusion steps for the prediction of all future time steps, where N, T_p is the diffusion step, prediction length, respectively; iii) Compared with current diffusion-based models for time series, DiffSTG 1) incorporates the graph as the condition so that the spatial correlations can be captured, and 2) is a non-autoregressive approach with $\widetilde{S} \times \widetilde{N}$ diffusion steps to get the estimated data distribution, where $\widetilde{S} = S/k < S$ and $\widetilde{N} = M < N$.

5 EXPERIMENTS

We conduct extensive experiments to evaluate the effectiveness of our proposed DiffSTG on three real-world datasets and compare it with other probabilistic baselines.

5.1 Experimental Setup

Datasets. In the experiments, we choose three real-world datasets from two domains, including a traffic flow dataset PEMS08 [32], and two air quality datasets AIR-BJ and AIR-GZ [46]. PEMS08 is a traffic flow dataset collected by the Caltrans Performance Measurement System (PeMS). It records the traffic flow recorded by sensors (nodes) deployed on the road network. In this work, we use the dataset extracted by STSGCN [32]. The traffic networks (adjacency matrix) for these datasets are constructed according to the actual road network. If the two sensors are on the same road, the two points are considered connected in the spatial network.

Table 1: Details of all datasets.

Dataset	Nodes	F	Data Type	Time interval	#Samples
PEMS08	170	1	Traffic flow	5 minutes	17,856
AIR-BJ	34	1	$PM_{2.5}$	1 hour	8,760
AIR-GZ	41	1	$PM_{2.5}$	1 hour	8,760

The air quality datasets AIR-BJ and AIR-GZ consist of one-year $PM_{2.5}$ readings collected by air quality monitoring stations in two metropolises (i.e., Beijing and Guangzhou) in China [46], respectively. AIR-BJ records data from 34 stations in Beijing from 2019/01/01 to 2019/12/31. And AIR-GZ records data from 41 stations in Guangzhou from 2017/01/01 to 2017/12/31. We build the spatial correlation matrix $\mathbf{A}$ using the distance between two stations. Statistics of the datasets are shown in Table 1.

Baselines. Both start-of-the-art probabilistic and deterministic models are included for the performance comparison.

Probabilistic baselines. The following methods are implemented as baselines for probabilistic STG forecasting:

- Latent ODE [29]. It defines a probabilistic generative process over time series from a latent initial state, which can be trained with variational inference.
- DeepAR [30], which utilizes a Gaussian distribution to model the data distribution;
- TimeGrad [25], which is an auto-regressive model that combines the diffusion model with an RNN-based encoder;
- CSDI [36], which is a diffusion-based non-autoregressive model first proposed for multivariate time series imputation. We mask all the future signals to adapt CSDI to our task.
- MC Dropout [43], which is developed based on MC Dropout [6] for probabilistic spatio-temporal forecasting.

Deterministic baselines. We choose some popular and state-of-the-art methods for comparison:

- DCRNN [17]: Diffusion Convolutional Recurrent Neural Network integrates diffusion convolution with sequence-to-sequence architecture to learn the representations of spatial dependencies and temporal relations.
- STGCN [47]: Spatial-Temporal Graph Convolution Network combines spectral graph convolution with 1D convolution to capture spatial and temporal correlations.
- STGNCDE [5]. Spatio-Temporal Graph Neural Controlled Differential Equation introduces two neural control differential equations (NCDE) for processing spatial and sequential data, respectively, which can be considered as an NCDE-based interpretation of graph convolutional networks.
- GMSDR [19]: Graph-based Multi-Step Dependency Relation improves RNN by explicitly taking the hidden states of multiple historical time steps as the input of each time unit.

Metrics. We choose the Continuous Ranked Probability Score (CRPS) [21] as the metric to evaluate the performance of probabilistic prediction, which is commonly used to measure the compatibility of an estimated probability distribution F with an observation x:

$$\mathrm{CRPS}(F, x) = \int_{\mathbb{R}} (F(z) - \mathbb{I}\{x \le z\})^2 \, \mathrm{d}z, \tag{19}$$

where $\mathbb{I}\{x \le z\}$ is an indicator function which equals one if $x \le z$, and zero otherwise. Smaller CRPS means better performance.

In addition, we leverage Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) to evaluate the performance of deterministic prediction. Let Y be the label, and $\hat{Y}$ denote the predictive result.

$$\mathrm{MAE}(Y, \hat{Y}) = \frac{1}{|Y|} \sum_{i=1}^{|Y|} \left| Y_i - \hat{Y}_i \right|, \tag{20}$$

$$\mathrm{RMSE}(Y, \hat{Y}) = \sqrt{\frac{1}{|Y|} \sum_{i=1}^{|Y|} \left(Y_i - \hat{Y}_i \right)^2}. \tag{21}$$

where a smaller metric means better performance. Among those metrics, CRPS is the primary metric of our focus to evaluate the performance of different probabilistic methods.

Implementation Details. As for hyperparameters, we set the batch size as 8 and use the Adam optimizer with a learning rate of 0.002, which is halved every 5 epochs. For CSDI and DiffSTG, we adopt the following quadratic schedule for variance schedule: $\beta_n = \left(\frac{N-n}{N-1} \sqrt{\beta_1} + \frac{n-1}{N-1} \sqrt{\beta_N} \right)^2$. We set the minimum noise level $\beta_1 = 0.0001$ and hidden size $C = 32$ and search the number of the diffusion step N and the maximum noise level β_N from a given parameter space ($N \in [50, 100, 200]$, and $\beta_N \in [0, 1, 0.2, 0.3, 0.4]$), and each model's best performance is reported in the experiment. For TimeGrad, as TimeGrad is an extremely time-consuming method, we followed the recommendations from the original paper and performed a search in a smaller diffusion setting space for the number of diffusion steps ($N \in [50, 100]$) and the maximum noise level ($\beta_N \in [0, 1, 0.2]$). This approach still allowed us to optimize TimeGrad's performance while considering the computational constraints. For other baselines, we utilize their codes and parameters in the original paper. For all datasets, the history length T_h, and prediction length T_p are both set to 12. All datasets are split into the training, validation, and test sets in chronological order with a ratio of 6:2:2. The models are trained on the training set and validated on the validation set by the early stopping strategy.

5.2 Performance Comparison

The efforts of stochastic models for probabilistic STG forecasting were traditionally scarce. Hence, we compare our model with state-of-the-art baselines in the field of probabilistic time series forecasting, including Latent ODE [29], DeepAR [30], TimeGrad [25], CSDI [36], and a recent STG probabilistic forecasting method MC Dropout [43]. We choose the Continuous Ranked Probability Score (CRPS) [21] as an evaluation metric. We also report MAE and RMSE of the deterministic forecasting results by averaging S (set to 8 in our paper) generated samples. Note that CRPS is the primary metric to evaluate the performance of those probabilistic methods.

In Table 2, DiffSTG outperforms all the probabilistic baselines: it reduces the CRPS by 5.6%, 4.3%, and 14.3% on the three datasets compared to the most competitive baseline in each dataset, respectively. Distributions in DeepAR and Latent ODE can be viewed as some types of low-rank approximations of the target, which naturally restricts their capability to model the true data distribution. TimeGrad outperforms LatentODE due to its DDPM-based architecture with tractable likelihoods that models the distribution in a general fashion. CSDI is a diffusion-based model originally proposed for time series imputation, thus performing worse in our forecasting tasks. MC Dropout achieves the second best performance on MAE and RMSE in most datasets, due to its strong ability in modeling the ST correlations. Our DiffSTG yields the best performance in both deterministic and probabilistic prediction, revealing that it can preserve the spatio-temporal learning capabilities of STGNNs as well as the uncertainty measurements of the diffusion models.

Table 2: Experiment results of probabilistic methods. Smaller MAE, RMSE, and CRPS indicate better performance. The best result is in bold, and the second-best result is underlined.

Method	AIR-BJ			AIR-GZ			PEMS08		
	MAE	RMSE	CRPS	MAE	RMSE	CRPS	MAE	RMSE	CRPS
Latent ODE [29]	20.61	32.27	0.47	12.92	18.76	0.30	26.05	39.50	0.11
DeepAR [30]	20.15	32.09	0.37	11.77	17.45	<u>0.23</u>	21.56	33.37	<u>0.07</u>
CSDI [36]	26.52	40.33	0.50	13.75	19.40	0.28	32.11	47.40	0.11
TimeGrad [25]	<u>18.64</u>	<u>31.86</u>	<u>0.36</u>	12.36	18.15	0.25	24.46	38.06	0.09
MC Dropout [43]	20.80	40.54	0.45	<u>11.12</u>	<u>17.07</u>	0.25	<u>19.01</u>	<u>29.35</u>	0.07
DiffSTG (ours)	**17.88**	**29.60**	**0.34**	**10.95**	**16.66**	**0.22**	**17.68**	**27.13**	**0.06**
Error reduction	-4.1%	-7.1%	-5.6%	-1.5%	-2.4%	-4.3%	-7.0%	-7.6%	-14.3%

Inference Time. Generally, diffusion-based forecasting models are much slower than other methods (due to their recurrent denoising nature), but they deliver promising performance, as introduced in TimeGrad and DDPM. To this end, here we mainly compare the inference speed of the diffusion-based forecasting method, including DiffSTG, TimeGrad, and CSDI. Table 3 reports the average time cost per prediction of the three diffusion-based forecasting models. We observe that TimeGrad is extremely time-consuming due to its recurrent architecture. DiffSTG (with M=100 and k=1) achieves 40× speed-up compared to TimeGrad, which stems from its non-autoregressive architecture. The accelerated sampling strategy achieves 3~4× speed-up beyond DiffSTG (M=100, k=1). We also find that when S is large, one can increase k for efficiency without loss of performance. See Sectioin 5.4 for more details.

Table 3: Time cost (by seconds) of diffusion-based models (TimeGrad, CSDI, and DiffSTG) in AIR-GZ ($T_h = 12, T_p = 12, N = 100$). S is the number of samples.

Method	$S = 8$	$S = 16$	$S = 32$
TimeGrad [25]	9.58	128.40	672.12
DiffSTG (M=100, k=1)	0.24	0.48	0.95
DiffSTG (M=40, k=1)	0.12	0.20	0.71
DiffSTG (M=40, k=2)	0.07	0.12	0.21
CSDI	0.51	0.88	1.82

Visualization. We plot the predicted distribution of different methods to investigate their performance intuitively. We choose the AIR-GZ and PEMS08 for demonstration, and more examples on other datasets can be found in Appendix A.3. We have the following observations: 1) Figure 5(a) shows that DiffSTG can capture the data distribution more precisely than DeepAR; 2) in Figure 5(b), where the predictions of both DeepAR and DiffSTG cover the observations, DiffSTG provides a more compact prediction interval, indicating the ability to provide more reliable estimations. 3) Note that the model also needs to learn to reconstruct the history in the loss of Eq. (14), we also illustrate the model's capability in history reconstruction in Figure 5(c); 4) Figure 5(d) draws the prediction result by a deterministic method STGCN [47] and DiffSTG. In the red box of Figure 5(d), the deterministic method fails to give an accurate prediction. In contrast, our DiffSTG model renders a bigger area (which covers the ground truth) in that region, indicating that the data therein is coupled with higher uncertainties. Such ability to accurately provide uncertainty can be of great help for practical decision-making; 5) Moreover, as shown in Figure 5(e), we illustrate the estimated distribution of DiffSTG on three stations, to illustrate its spatial dependency learning ability. Compared with station 29, the estimated distribution of station 4 is more similar to station 1, which is reasonable because the air quality of a station has stronger connections with its nearby neighbors. Equipped with the proposed denoising network UGnet, the model is able to capture the ST correlations, leading to more reliable and accurate estimation.

5.3 Ablation Study

We conduct an ablation study on the AIR-GZ dataset to verify the effect of each component. We design three variants of DiffSTG and compare them with the full version of DiffSTG. The differences between these four models are described as follows:

- **(w/o Spatial)**: removing the GNN component in UGnet, i.e., without modeling the spatial correlation.
- **(w/o Temporal)**: removing the TCN component in UGnet, i.e., without modeling the temporal correlation.
- **w/o U-structure**: removing the Unet-based structure in UGnet and only using one TCN for feature extraction.
- **DiffSTG**: the final version of the proposed model.

Figure 4 illustrates the results. We have the following observations: Firstly, removing the spatial dependency learning in UGnet (w/o Spatial) brings considerable degeneration, which validates the importance of modeling spatial correlations between nodes. Secondly, when turning off the temporal dependency learning in UGnet (w/o Temporal), the performance drops significantly on all evaluation metrics. Thirdly, when detaching the Unet-based structure in UGnet, the performance degrades dramatically, which

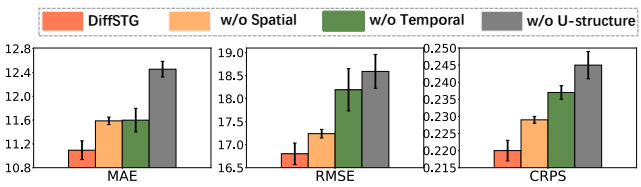


Figure 4: Ablation study.

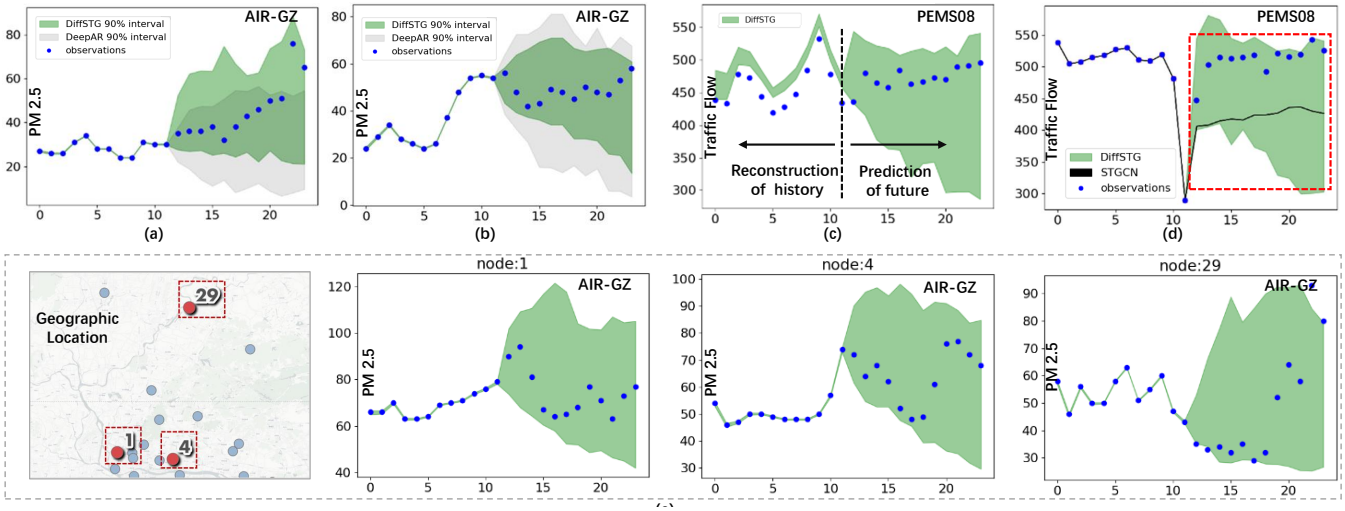


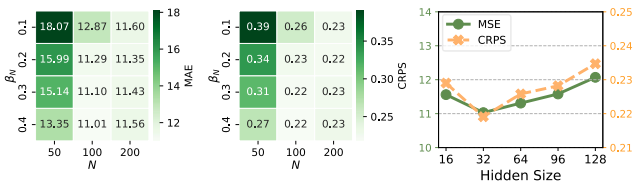
Figure 5: Example of probabilistic spatio-temporal graph forecasting for air quality and traffic dataset.

demonstrates the merits of an Unet-based structure in capturing ST-dependencies at different granularities.

5.4 Hyperparameter Study

In this section, we examine the impact of several crucial hyperparameters on DiffSTG. Specifically, we report i) the performance on AIR-GZ under different variance schedules (i.e., the combination of β_N and diffusion step N) and hidden size C. ii) influence of kernel size in TCN; iii) the number of generated samples S and the number of utilized samples k in the proposed sampling strategy.

Effect of the Variance Schedule and Hidden Size. For different variance schedules $\{\beta_1, \ldots, \beta_N\}$, we set $\beta_1 = 0.0001$ and let β_N and N be from two search spaces, where $N \in [50, 100, 200]$ and $\beta_N \in [0.1, 0.2, 0.3, 0.4]$. A variance schedule can be specified by a combination of β_N and N. The results are shown in Figure 6. We note that the performance deteriorates rapidly when $N = 50$ and $\beta_N = 0.1$. In this case, the result of the forward process is far away from a Gaussian distribution. Consequently, the reverse process starting with Gaussian distribution becomes an inaccurate approximation, which heavily injures the model performance. When N gets larger, there is a higher chance of getting a promising result.



(a) The effect of variance schedule (b) The effect of hidden size

Figure 6: Influence of hyperparameters.

Figure 6 shows the results of DiffSTG with $N = 100$, and $\beta_N = 0.4$ vs. different hidden size C, from which we observe that the performance first slightly increases and then drops with the increase in hidden size. Compared with the variance schedule, the model's performance is much less sensitive to the hidden size.

Influence of Kernel Size in TCN. We report the influence of kernel size ($K = 2, 3, 4, 5$) in TCN as in Table 4. Overall, we can see that the kernel size in TCN has a small impact on the performance of the model. In particular, a kernel size of 3 appears to be a good choice when the input and prediction horizon is 12.

Table 4: Influence of the kernel size K in TCN on AIR-GZ ($T_h = 12, T_p = 12, N = 100, \beta_N = 0.1$).

Metric	K=2	K=3	K=4	K= 5
MAE	13.85	13.38	14.02	14.86
RMAE	19.73	19.25	19.95	20.99
CRPS	0.27	0.26	0.28	0.29

Effect of the Number of Generated Samples S. We investigate the relationship between the number of samples S and the performance in Figure 7. It shows the effect of probabilistic forecasting, as well as the effect on deterministic forecasting. From the figure, we can see that five or ten samples are enough to estimate good distributions. While increasing the number of samples further improves the performance, the improvement becomes marginal over 32 samples.

Effect of k. Recall that k is the number of utilized samples in the last few diffusion steps when sampling S samples. We provide the results of $k = 1$ and $k = 2$ in Figure 7, in which we have several interesting observations: i) When S is large enough (i.e., $S > 32$), the performance of $k = 2$ is almost the same as $k = 1$, and the sample speed of $k = 2$ is 1.5 times faster than $k = 1$; ii) When the number of reverse diffusion processes (i.e., S/k) is settled, a large k can increase sample diversity thus leading to better performance, especially when S is small.

In light of the above results, we give the following recommendations for the combination of S and k: 1) when S is small, a small k

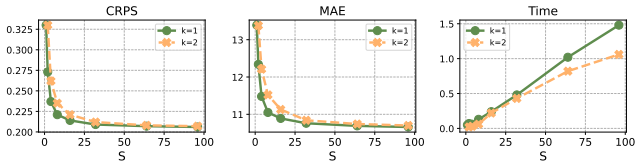


Figure 7: The effect of the number of generated samples and the influence of k.

is recommended to increase the sample diversity for better performance; 2) when S is large, one can increase k for efficiency without much lose of the performance.

5.5 Limitations

Though promising in probabilistic prediction, DiffSTG still has a performance gap compared with current state-of-the-art STGNNs in terms of deterministic forecasting. Table 5 shows the deterministic prediction performance of DiffSTG (by averaging 8 in generate samples) and four deterministic methods, including DCRNN [17], STGCN [47], STGNCDE [5], and GMSDR [19]. While DiffSTG is competitive with most probabilistic methods, it is still inferior to the state-of-the-art deterministic methods. Different from deterministic methods, the optimization goal of DiffSTG is derived from a variational inference perspective (see details in Appendix. A.1), where the learned posterior distribution might be inaccurate when the data samples are insufficient. We have similar observations in other DDPM-based models, such as TimeGrad and CSDI, as shown Table 2.

Though, we would like to emphasize a key advantage of probabilistic such as DiffSTG, is its ability to provide probabilistic forecasting. This is the starting point of our focus and an important aspect that even the SOTA GNN-based methods may not be able to offer. We leave improving DiffSTG to surpass those deterministic methods in future work.

Table 5: Comparison with deterministic methods. Lower MAE and RMSE indicate better performance.

Method	AIR-BJ		AIR-GZ		PEMS08	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
DCRNN	16.99	**28.00**	10.23	15.21	18.56	28.73
STGCN	19.54	30.51	11.05	16.54	20.15	30.14
STGNCDE	19.17	29.56	10.51	16.11	**15.83**	25.05
GMSDR	**16.60**	28.50	**9.72**	**14.55**	16.01	**24.84**
DiffSTG	17.88	29.60	11.04	16.75	17.68	27.13

6 RELATED WORK

Spatio-temporal Graph Forcasting. Recently, a large body of research has been studied on spatio-temporal forecasting in different scenarios, such as traffic forecasting [7, 10, 24, 44, 47] and air quality forecasting [18]. STGNNs have become dominant models in this field, which combine GNN and temporal components (e.g., TCN and RNN) to capture the spatial correlations and temporal features, respectively. However, most existing works focus on point estimation while ignoring quantifying the uncertainty of predictions. To fill this gap, this paper develops a conditional diffusion-based method that couples the spatio-temporal learning capabilities of STGNNs with the uncertainty measurements of diffusion models.

Score-based Generative Models. The diffusion model that we adopt belongs to score-based generative models (please refer to Section 2 for more details), which learn the gradient of the log-density with respect to the inputs, called Stein Score function [9, 39]. At inference time, they use the gradient estimate to sample the data via Langevin dynamics [34]. By perturbing the data through different noise levels, these models can capture both coarse and fine-grained features in the original data. Which, leads to their impressive performance in many domains, such as image [8], audio [4, 14], graph [22] and time series [25, 36].

Time Series Forecasting. Methods in time series forecasting can be classified into two streams. The first one is deterministic methods, including transformer-based approaches [42, 48, 50] and RNN-based models [3]. The second one is probabilistic methods [23, 30], which aims to provide stochastic predictions for future time series. A rising trend in this stream is leveraging the diffusion-based models to improve the performance of probabilistic predictions [2, 16, 25, 41]. Though those methods have achieved promising performance, they still lack the ability to model the spatial correlations of different nodes if applied to STG forecasting. The above limitation also motivates us to develop a more powerful diffusion-based model for STG that can effectively capture spatial correlations.

7 CONCLUSION AND FUTURE WORK

In this paper, we first propose a novel probabilistic framework called DiffSTG for spatio-temporal graph forecasting. To the best of our knowledge, this is the first work that generalizes the DDPM to spatio-temporal graphs. DiffSTG combines the spatio-temporal learning capabilities of STGNNs with the uncertainty measurements of diffusion models. Unlike previous diffusion-based models designed for the image or sequential data, we devise the first denoising network UGnet for capturing the spatial and temporal correlations in STG data. By leveraging an Unet-based architecture to capture multi-scale temporal dependencies and a Graph Neural Network (GNN) to model spatial correlations, UGnet offers a powerful new member to the DDPM denoising network family, tailored for STG data. Extensive experiments demonstrate the effectiveness and efficiency of our proposed method.

Furthermore, we accelerate the training and inference speed by elaborately designed non-autoregressive prediction architecture and a sample acceleration strategy. This improvement in efficiency greatly benefits the potential of diffusion-based methods for STG forecasting tasks, making them more viable and appealing for practical applications.

Some interesting directions are worth exploring in the future. Since we only use the vanilla GCN in the denoising network for the model's simplicity, it remains open to incorporating more powerful graph neural networks to better capture the ST dependencies in the data. Another direction is to apply DiffSTG to other spatio-temporal learning tasks, such as spatio-temporal graph imputation.

REFERENCES

[1] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271* (2018).

[2] Ping Chang, Huayu Li, Stuart F Quan, Janet Roveda, and Ao Li. 2023. TDSTF: Transformer-based Diffusion probabilistic model for Sparse Time series Forecasting. *arXiv preprint arXiv:2301.06625* (2023).

[3] Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. 2018. Recurrent neural networks for multivariate time series with missing values. *Scientific reports* 8, 1 (2018), 6085.

[4] Nanxin Chen, Yu Zhang, Heiga Zen, Ron J Weiss, Mohammad Norouzi, and William Chan. 2020. WaveGrad: Estimating Gradients for Waveform Generation. In *International Conference on Learning Representations*.

[5] Jeongwhan Choi, Hwangyong Choi, Jeehyun Hwang, and Noseong Park. 2022. Graph neural controlled differential equations for traffic forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 6367–6374.

[6] Yarin Gal, Jiri Hron, and Alex Kendall. 2017. Concrete dropout. *Advances in neural information processing systems* 30 (2017).

[7] Shengnan Guo, Youfang Lin, Huaiyu Wan, Xiucheng Li, and Gao Cong. 2021. Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting. *IEEE Transactions on Knowledge and Data Engineering* (2021).

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems* 33 (2020), 6840–6851.

[9] Aapo Hyvärinen and Peter Dayan. 2005. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research* 6, 4 (2005).

[10] Jiahao Ji, Jingyuan Wang, Zhe Jiang, Jiawei Jiang, and Hu Zhang. 2022. STDEN: Towards physics-guided neural networks for traffic flow prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 4048–4056.

[11] Jaehyeon Kim, Sungwon Kim, Jungil Kong, and Sungroh Yoon. 2020. Glow-tts: A generative flow for text-to-speech via monotonic alignment search. *Advances in Neural Information Processing Systems* 33 (2020), 8067–8077.

[12] Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).

[13] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.

[14] Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. 2020. DiffWave: A Versatile Diffusion Model for Audio Synthesis. In *International Conference on Learning Representations*.

[15] Lixin Li and Peter Revesz. 2004. Interpolation methods for spatio-temporal geographic data. *Computers, Environment and Urban Systems* 28, 3 (2004), 201–227.

[16] Yan Li, Xinjiang Lu, Yaqing Wang, and Dejing Dou. 2023. Generative Time Series Forecasting with Diffusion, Denoise, and Disentanglement. *arXiv preprint arXiv:2301.03028* (2023).

[17] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations*.

[18] Yuxuan Liang, Yutong Xia, Songyu Ke, Yiwei Wang, Qingsong Wen, Junbo Zhang, Yu Zheng, and Roger Zimmermann. 2022. AirFormer: Predicting Nationwide Air Quality in China with Transformers. *arXiv preprint arXiv:2211.15979* (2022).

[19] Dachuan Liu, Jin Wang, Shuo Shang, and Peng Han. 2022. Msdr: Multi-step dependency relation networks for spatial temporal forecasting. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1042–1050.

[20] Jinglin Liu, Chengxi Li, Yi Ren, Feiyang Chen, and Zhou Zhao. 2022. Diffsinger: Singing voice synthesis via shallow diffusion mechanism. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 11020–11028.

[21] James E Matheson and Robert L Winkler. 1976. Scoring rules for continuous probability distributions. *Management science* 22, 10 (1976), 1087–1096.

[22] Chenhao Niu, Yang Song, Jiaming Song, Shengjia Zhao, Aditya Grover, and Stefano Ermon. 2020. Permutation invariant graph generation via score-based generative modeling. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4474–4484.

[23] Soumyasundar Pal, Liheng Ma, Yingxue Zhang, and Mark Coates. 2021. RNN with particle flow for probabilistic spatio-temporal forecasting. In *International Conference on Machine Learning*. PMLR, 8336–8348.

[24] Hao Peng, Hongfei Wang, Bowen Du, Md Zakirul Alam Bhuiyan, Hongyuan Ma, Jianwei Liu, Lihong Wang, Zeyu Yang, Linfeng Du, Senzhang Wang, et al. 2020. Spatial temporal incidence dynamic graph neural networks for traffic flow forecasting. *Information Sciences* 521 (2020), 277–290.

[25] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. 2021. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning*. PMLR, 8857–8868.

[26] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. 2014. Stochastic backpropagation and approximate inference in deep generative models. In *International conference on machine learning*. PMLR, 1278–1286.

[27] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10684–10695.

[28] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*. Springer, 234–241.

[29] Yulia Rubanova, Ricky TQ Chen, and David K Duvenaud. 2019. Latent ordinary differential equations for irregularly-sampled time series. *Advances in neural information processing systems* 32 (2019).

[30] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* 36, 3 (2020), 1181–1191.

[31] Jelena Simeunović, Baptiste Schubnel, Pierre-Jean Alet, and Rafael E Carrillo. 2021. Spatio-temporal graph neural networks for multi-site PV power forecasting. *IEEE Transactions on Sustainable Energy* 13, 2 (2021), 1210–1220.

[32] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. 2020. Spatial-Temporal Synchronous Graph Convolutional Networks: A New Framework for Spatial-Temporal Network Data Forecasting. *AAAI 2020: The Thirty-Fourth AAAI Conference on Artificial Intelligence* 34, 1 (2020), 914–921.

[33] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations*.

[34] Yang Song and Stefano Ermon. 2019. Generative modeling by estimating gradients of the data distribution. *Advances in Neural Information Processing Systems* 32 (2019).

[35] Yang Song and Stefano Ermon. 2020. Improved techniques for training score-based generative models. *Advances in neural information processing systems* 33 (2020), 12438–12448.

[36] Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. 2021. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. *Advances in Neural Information Processing Systems* 34 (2021), 24804–24816.

[37] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. 2016. WaveNet: A Generative Model for Raw Audio. In *9th ISCA Speech Synthesis Workshop*. 125–125.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[39] Pascal Vincent. 2011. A connection between score matching and denoising autoencoders. *Neural computation* 23, 7 (2011), 1661–1674.

[40] Vikram Voleti, Alexia Jolicoeur-Martineau, and Christopher Pal. 2022. MCVD-Masked Conditional Video Diffusion for Prediction, Generation, and Interpolation. In *Advances in Neural Information Processing Systems*.

[41] Zhixian Wang, Qingsong Wen, Chaoli Zhang, Liang Sun, and Yi Wang. 2023. DiffLoad: Uncertainty Quantification in Load Forecasting with Diffusion Model. *arXiv preprint arXiv:2306.01001* (2023).

[42] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2023. Transformers in time series: A survey. In *International Joint Conference on Artificial Intelligence (IJCAI)*.

[43] Dongxia Wu, Liyao Gao, Matteo Chinazzi, Xinyue Xiong, Alessandro Vespignani, Yi-An Ma, and Rose Yu. 2021. Quantifying Uncertainty in Deep Spatiotemporal Forecasting. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1841–1851.

[44] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. In *IJCAI 2019: 28th International Joint Conference on Artificial Intelligence*. 1907–1913.

[45] Huaxiu Yao, Fei Wu, Jintao Ke, Xianfeng Tang, Yitian Jia, Siyu Lu, Pinghua Gong, Jieping Ye, and Zhenhui Li. 2018. Deep multi-view spatial-temporal network for taxi demand prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.

[46] Xiuwen Yi, Junbo Zhang, Zhaoyuan Wang, Tianrui Li, and Yu Zheng. 2018. Deep distributed fusion network for air quality prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 965–973.

[47] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *IJCAI 2018: 27th International Joint Conference on Artificial Intelligence*. 3634–3640.

[48] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.

[49] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81.

[50] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*. PMLR, 27268–27286.

A APPENDIX

A.1 Details of DDPM

We introduce the details of denoising diffusion probabilistic models in this section.

Diffusion probabilistic models are latent variable models that consist of two processes, namely the forward process and the reverse process. The forward process is a fixed Gaussian transition process as defined in Eq. (1) and Eq. (2). The reverse process is a learnable Gaussian transition process defined in Eq. (4) and Eq. (5). Then, the parameters θ are learned by minimizing the negative log-likelihood via the variational lower bound (ELBO):

$$\begin{aligned}
&\min_\theta \mathbb{E}_{q(\boldsymbol{x}_0)}\left[-\log p_\theta(\boldsymbol{x}_0)\right] \\
&\leq -\log p_\theta\left(\boldsymbol{x}_0\right)+D_{\mathrm{KL}}\left(q\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right) \| p_\theta\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right)\right) \\
&=-\log p_\theta\left(\boldsymbol{x}_0\right)+\mathbb{E}_{\boldsymbol{x}_{1:N} \sim q\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right)}\left[\log \frac{q\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_{0:N}\right) / p_\theta\left(\boldsymbol{x}_0\right)}\right] \\
&=-\log p_\theta\left(\boldsymbol{x}_0\right)+\mathbb{E}_q\left[\log \frac{q\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_{0:N}\right)}+\log p_\theta\left(\boldsymbol{x}_0\right)\right] \\
&=\mathbb{E}_{q\left(\boldsymbol{x}_{0:N}\right)}\left[\log \frac{q\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_{0:N}\right)}\right] := \mathcal{L}_{\mathrm{ELBO}}.
\end{aligned} \tag{22}$$

We can further decompose $\mathcal{L}_{\mathrm{ELBO}}$ into different terms according to the property of Markov chains:

$$\begin{aligned}
&\mathcal{L}_{\mathrm{ELBO}} \\
&=\mathbb{E}_{q\left(\boldsymbol{x}_{0:N}\right)}\left[\log \frac{q\left(\boldsymbol{x}_{1:N} \mid \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_{0:N}\right)}\right] \\
&=\mathbb{E}_q\left[\log \frac{\prod_{n=1}^N q\left(\boldsymbol{x}_n \mid \boldsymbol{x}_{n-1}\right)}{p_\theta\left(\boldsymbol{x}_N\right) \prod_{n=1}^N p_\theta\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n\right)}\right] \\
&=\mathbb{E}_q\left[-\log p_\theta\left(\boldsymbol{x}_N\right)+\sum_{t=2}^N \log \frac{q\left(\boldsymbol{x}_n \mid \boldsymbol{x}_{n-1}\right)}{p_\theta\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n\right)}+\log \frac{q\left(\boldsymbol{x}_1 \mid \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_0 \mid \boldsymbol{x}_1\right)}\right] \\
&=\mathbb{E}_q\left[\log \frac{q\left(\boldsymbol{x}_N \mid \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_N\right)}+\sum_{t=2}^N \log \frac{q\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n, \boldsymbol{x}_0\right)}{p_\theta\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n\right)}-\log p_\theta\left(\boldsymbol{x}_0 \mid \boldsymbol{x}_1\right)\right] \\
&=\mathbb{E}_q[\underbrace{D_{\mathrm{KL}}\left(q\left(\boldsymbol{x}_N \mid \boldsymbol{x}_0\right) \| p_\theta\left(\boldsymbol{x}_N\right)\right)}_{\mathcal{L}_N} \\
&+\sum_{t=2}^N \underbrace{D_{\mathrm{KL}}\left(q\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n, \boldsymbol{x}_0\right) \| p_\theta\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n\right)\right)}_{\mathcal{L}_{n-1}} \\
&\underbrace{-\log p_\theta\left(\boldsymbol{x}_0 \mid \boldsymbol{x}_1\right)}_{\mathcal{L}_0}].
\end{aligned} \tag{23}$$

By the property in Eq. (3), [8] show that the forward process posterior when conditioned on $\boldsymbol{x}_0$, i.e., $q(\boldsymbol{x}_{n-1}|\boldsymbol{x}_n, \boldsymbol{x}_0)$ is tractable, formulated as

$$q\left(\boldsymbol{x}_{n-1} \mid \boldsymbol{x}_n, \boldsymbol{x}_0\right)=\mathcal{N}\left(\boldsymbol{x}_{n-1} ; \tilde{\boldsymbol{\mu}}\left(\boldsymbol{x}_n, \boldsymbol{x}_0\right), \tilde{\beta}_n \mathbf{I}\right) \tag{24}$$

where

$$\tilde{\mu}_n\left(\boldsymbol{x}_n, \boldsymbol{x}_0\right)=\frac{\sqrt{\alpha_{n-1}} \beta_n}{1-\alpha_n} \boldsymbol{x}_0+\frac{\sqrt{\alpha_n}\left(1-\alpha_{n-1}\right)}{1-\alpha_n} \boldsymbol{x}_n, \tag{25}$$

and

$$\tilde{\beta}_n=\frac{1-\alpha_{n-1}}{1-\alpha_n} \beta_n. \tag{26}$$

So far, we can see that each term in $\mathcal{L}_{\mathrm{ELBO}}$ (except for $\mathcal{L}_0$) calculates the KL Divergence between two Gaussian distributions, therefore they can be computed in closed form. $\mathcal{L}_N$ is constant that can be ignored in training because q has no learnable parameters and $\boldsymbol{x}_N$ is a Gaussian noise. [8] models $\mathcal{L}_0$ using a separate discrete decoder. Especially, the loss term of $\mathcal{L}_t$ ($t \in \{2, \cdots, T\}$), have the following closed form:

$$\mathbb{E}_{\boldsymbol{x}_0, \epsilon}\left[\frac{\beta_n^2}{2 \Sigma_\theta \alpha_n\left(1-\alpha_n\right)}\left\|\epsilon-\epsilon_\theta\left(\sqrt{\alpha_n} \boldsymbol{x}_0+\sqrt{1-\alpha_n} \epsilon, n\right)\right\|^2\right],$$

which can be further simplified by removing the coefficient in the loss term, formulated as

$$\mathbb{E}_{\boldsymbol{x}_0, \epsilon}\left[\left\|\epsilon-\epsilon_\theta\left(\sqrt{\alpha_n} \boldsymbol{x}_0+\sqrt{1-\alpha_n} \epsilon, n\right)\right\|^2\right].$$

A.2 Details of UGnet

We propose a novel denoising network to effectively capture spatio-temporal correlations in STG data, named UGnet. It adopts an Unet-like architecture in the temporal dimension and can also process the graph as the condition. The Unet structure can capture features at different levels because its Convolutional Neural Networks (CNN) kernels gradually merge low-level features into high-level features. Similarly, in the context of spatio-temporal forecasting, we naturally have different granularities in the temporal dimension (e.g., 15 minutes, 30 minutes, and 1 hour). Therefore, an intuitive way is to adopt the idea in Unet that gradually reduce the shape in the temporal dimension and reverse it back so that temporal features at different levels can be well captured. Doing so also brings the model the ability to scale up to large STG.

Specifically, as shown in Figure 8, UGnet $\boldsymbol{\epsilon}_\theta(\mathcal{X}^{\mathrm{all}} \times \mathbb{R} | \mathcal{X}^{\mathrm{all}}_{\mathrm{msk}}, \mathcal{G}) \rightarrow \mathcal{X}^{\mathrm{all}}$ takes $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}}$, $\boldsymbol{x}^{\mathrm{all}}_n$, n, $\mathcal{G}$ as input, and outputs the denoised noise $\boldsymbol{\epsilon}$. We first concatenate $\boldsymbol{x}^{\mathrm{all}}_n \in \mathbb{R}^{F \times V \times T}$ and $\boldsymbol{x}^{\mathrm{all}}_{\mathrm{msk}} \in \mathbb{R}^{F \times V \times T}$ in the temporal dimension to form a new matrix $\widetilde{\boldsymbol{x}}^{\mathrm{all}}_n \in \mathbb{R}^{F \times V \times 2T}$. UGnet contains several Spatio-temporal Residual Blocks (ST-Residual Block for short) with two types: the down-residual block and the up-residual block. The down-residual blocks gradually reduce the shape in the temporal dimension (i.e., increase the temporal granularity). While the up-residual blocks gradually convert the temporal granularity back to the level that is the same as the input. Both blocks contain the same residual block that can capture both spatial and temporal correlations in the data with the help of the graph structure.

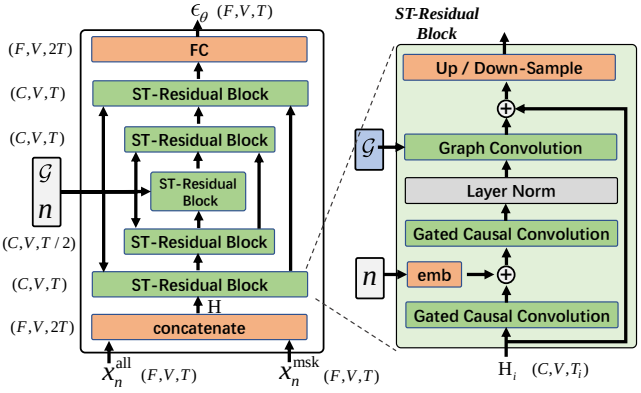


Figure 8: The architecture of denoising network UGnet. It adopts an Unet-like structure to model both spatial and temporal dependencies at different temporal granularities, conditioned on the noise level and given graph structure.

Here, we introduce the details of the ST-Residual block. We first project input $\boldsymbol{x}_n^{\text{all}} \in \mathbb{R}^{F\times V\times T}$ into a high-dimensional representation $\mathbf{H} \in \mathbb{R}^{C\times V\times 2T}$ by a linear layer, where C is the projected dimension. Let $\mathbf{H}_i \in \mathbb{R}^{C\times V\times T_i}$ denote the input of the i-th ST-Residual Block, where T_i is the length of the time dimension, and $\mathbf{H}_0 = \mathbf{H}$.

Temporal Dependence Modeling. As shown in Figure 8, $\mathbf{H}_i$ is first fed into a Temporal Convolution Network (TCN) [1] for modeling temporal dependence, which is a 1-D gated causal convolution with K kernel size with padding to get the same shape with input. The convolution kernel $\Gamma_{\mathcal{T}} \in \mathbb{R}^{K\times C_{\text{in}}^{\text{t}}\times C_{\text{out}}^{\text{t}}}$ maps the input $\mathbf{H}_i$ to two outputs $\mathbf{P}_i, \mathbf{Q}_i$ with the same shape $\mathbf{P}_i/\mathbf{Q}_i \in \mathbb{R}^{C_{\text{out}}^{\text{t}}\times V\times T_i}$. As a result, the temporal gated convolution can be defined as,

$$\Gamma_{\mathcal{T}}(\mathbf{H}_i) = \mathbf{P}_i \odot \sigma(\mathbf{Q}_i) \in \mathbb{R}^{C_{\text{out}}^{\text{t}}\times V\times T_i} := \overline{\mathbf{H}}_i, \tag{27}$$

where $\odot$ is the element-wise Hadamard product, and σ is the sigmoid activation function of GLU. The item $\sigma(\mathbf{Q}_i)$ can be considered a gate that filters the useful information of $\mathbf{P}_i$ into the next layer. Furthermore, residual connections are implemented among stacked temporal convolutional layers to further exploit the full input times horizon.

Spatial Dependence Modeling. Graph convolution network (GCN) is employed to directly extract highly meaningful features and patterns in the space domain. The input of GCN is the node feature matrix, which is reshaped output of the TCN layer in our case, denoted as $\overline{\mathbf{H}}_i \in \mathbb{R}^{V\times C_{\text{in}}^{\text{g}}}$, where $C_{\text{in}}^{\text{g}} = T_i \times C_{\text{out}}^{\text{t}}$). A general formulation [49] of a graph convolution can be denoted as

$$\Gamma_{\mathcal{G}}(\overline{\mathbf{H}}_i) = \sigma\left(\Phi\left(\mathbf{A}_{\text{gcn}}, \overline{\mathbf{H}}_i\right)\mathbf{W}_i\right), \tag{28}$$

where $\mathbf{W}_i \in \mathbb{R}^{C_{\text{in}}^{\text{g}}\times C_{\text{in}}^{\text{g}}}$ denotes a trainable parameter and σ is an activation function. $\Phi(\cdot)$ is an aggregation function that decides the rule of how neighbors' features are aggregated into the target node. In our work, we do not focus on how to develop an elaborately designed function $\Phi(\cdot)$. Therefore, we use the form in the most popular vanilla GCN [13] that defines a symmetric normalized summation function as $\Phi_{\text{gcn}}\left(\mathbf{A}_{\text{gcn}}, \overline{\mathbf{H}}_i\right) = \mathbf{A}_{\text{gcn}}\overline{\mathbf{H}}_i$, where $\mathbf{A}_{\text{gcn}} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{A}+\mathbf{I})\mathbf{D}^{-\frac{1}{2}} \in \mathbb{R}^{V\times V}$ is a normalized adjacent matrix. $\mathbf{I}$ is the identity matrix and $\mathbf{D}$ is the diagonal degree matrix with $\mathbf{D}_{ii} = \sum_j(\mathbf{A}+\mathbf{I})_{ij}$.

We report the parameter number of probabilistic models below:

Table 6: Parameter number.

Method	TimeGrad	CSDI	DiffSTG	DeepAR	LatentODE	MC Dropout
number	52,423	48,289	149,305	25,474	24,073	371,393

A.3 Additional Prediction Examples

This section illustrates various probabilistic forecasting examples to show the characteristic of different methods. Note that the scales of the y-axis depend on the stations.

We compare DiffSTG with TimeGrad for selected stations of AIR-BJ in Figure 10. The geographic distribution of stations is shown in Figure 9. We select two groups of nodes according to their spatial location. Nodes in the first group are far away from each other, including nodes 0, 2, 17, and 20. While nodes in the second group are close to each other, including nodes 7, 8, 9, and 18.

For the comparison in Figure 10, while TimeGrad fails to capture the data distribution, DiffSTG computes reasonable probabilistic forecasting for a majority of the stations. And we can see that DiffSTG tends to provide similar estimated distribution for stations nearby, which is reasonable since the air quality of a station is strongly correlated with its neighbors. The above examples further illustrate that DiffSTG can effectively learn the spatial and temporal dependency in STG, thus providing more reliable and accurate estimations than others.

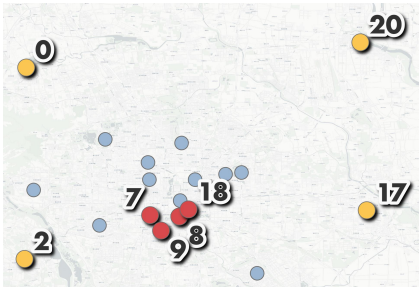


Figure 9: Geographic distribution of stations on AIR-BJ.

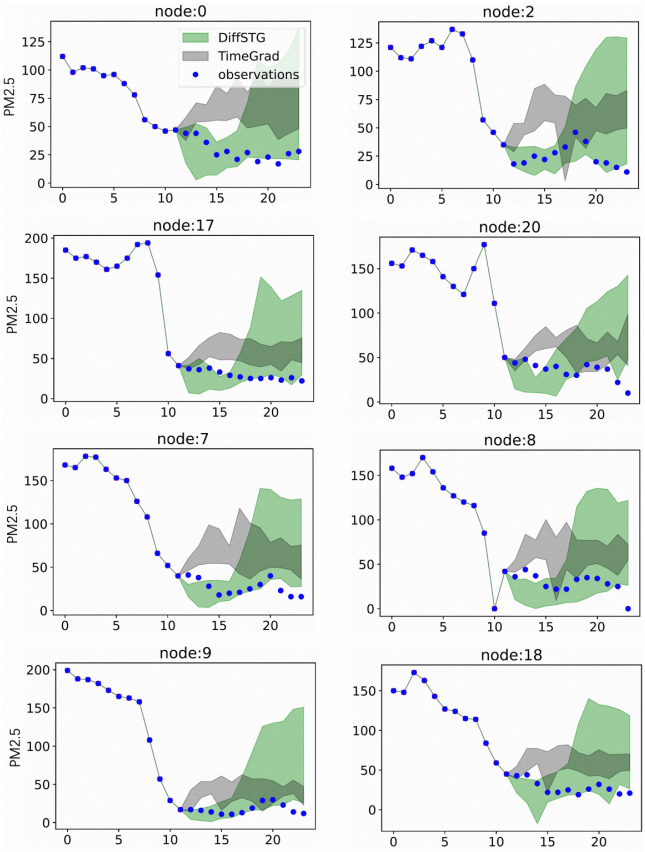


Figure 10: Comparison of probabilistic STG forecasting between TimeGrad and DiffSTG for air quality dataset (AIR-BJ).